

AGENTIC SOFTWARE ENGINEERING SYSTEM

URL Shortener – Assessment Solution

Production-Grade Prototype | Stateful Orchestration | Controlled Autonomy

1. Executive Summary

This solution proposes a Governed Agentic Software Engineering System that transforms a natural-language requirement into a reviewable engineering outcome. The URL Shortener is the demonstration workload, while the agentic orchestration layer is the primary engineering capability.

The platform coordinates the full SDLC—requirements, architecture/design, implementation, testing, documentation and release readiness—using an explicit dependency graph and persistent workflow state. It supports sequential and parallel execution, synchronization, context and decision lineage, human approval checkpoints, bounded retries, fallback, rollback, safe-stop, policy guardrails, auditability, reliability metrics and dynamic replanning.

2. Assessment Objectives and Design Response

Assessment Area	Design Response	Demonstrable Evidence
Requirement Understanding	Requirement Intelligence Agent	Normalized requirements, ambiguity, assumptions, acceptance criteria
Task Decomposition	Planning Agent + executable dependency graph	Tasks, dependencies, sequencing and gates
Brownfield Reasoning	Codebase Analysis + Impact Analysis	Impacted modules, services, APIs and data flows
Workflow Orchestration	Stateful graph orchestrator	Parallel paths, synchronization, gates and re-planning
Engineering Output	Specialized engineering agents	Code, API/schema, tests and documentation
Validation/Risk	Validation + Security + Policy Engine	Quality gates, risk register and safety controls
Controlled Autonomy	Permission model + Approval Manager	High-impact approval workflow
Final Summary	Documentation + Release Readiness agents	Engineering summary, artifacts, risks and limitations

3. Solution Principles

- The orchestrator, not the individual LLM agent, is the control plane.
- Agents have explicit responsibilities, tools, permissions and autonomy boundaries.
- Workflow execution is graph-based rather than a simple linear chain.
- Every meaningful output is represented as a versioned artifact.
- High-impact actions are policy-controlled and can require human approval.
- Failures are expected and handled through bounded recovery mechanisms.
- Requirement or upstream artifact changes trigger targeted impact analysis and replanning.
- Humans retain oversight, approvals and final quality ownership.

4. High-Level Architecture

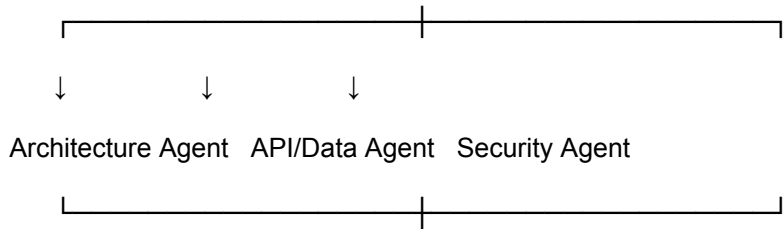
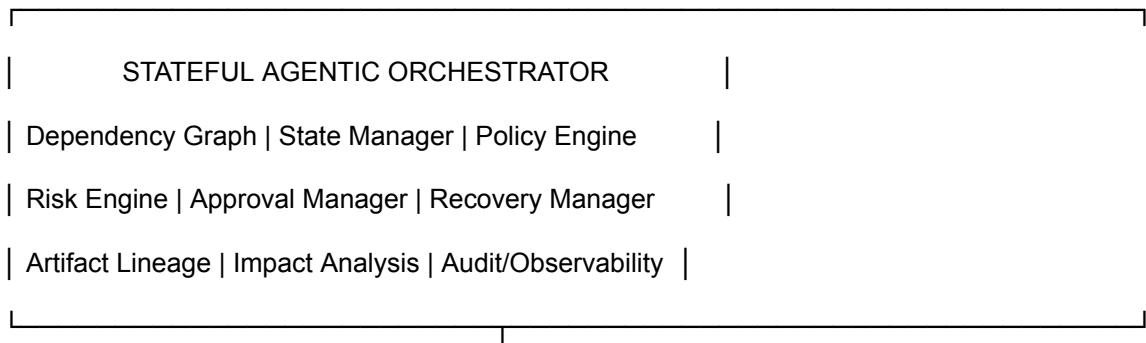
USER REQUIREMENT



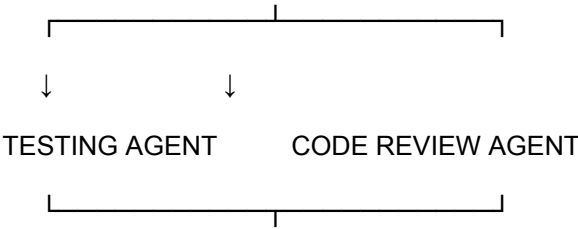
REQUIREMENT INTELLIGENCE AGENT



PLANNING / DECOMPOSITION AGENT



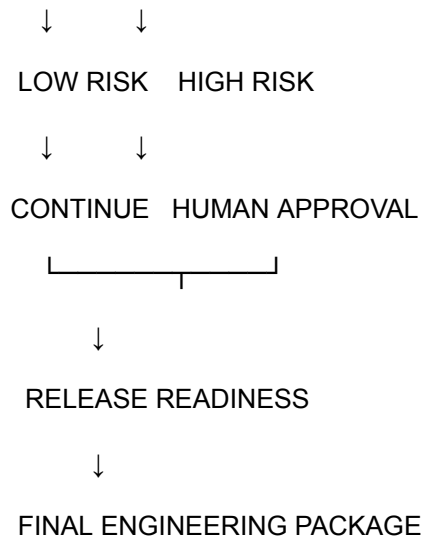
IMPLEMENTATION AGENT



VALIDATION AGENT



RISK EVALUATION



5. Agent Responsibilities

Agent	Responsibility
Requirement Intelligence Agent	Interprets intent, identifies ambiguity, normalizes requirements, records assumptions and acceptance criteria.
Planning Agent	Converts requirements into actionable tasks, dependencies, sequencing, risk and execution strategy.
Codebase Analysis Agent	For brownfield work, maps modules, services, APIs, data flows and likely impact.
Architecture Agent	Produces architecture, component boundaries, data flow, reliability and scalability decisions.
API/Data Agent	Defines API contracts, schemas, indexes, constraints and persistence decisions.
Implementation Agent	Creates or modifies production-quality code and configuration.
Testing Agent	Creates unit, integration, edge-case and failure tests.
Security Agent	Evaluates input validation, access control, abuse risks, secrets and security controls.
Code Review Agent	Reviews correctness, maintainability, design quality and standards.
Validation Agent	Checks requirements, implementation, tests, security and release gates.
Documentation Agent	Creates setup, API, architecture and operational documentation.
Release Readiness Agent	Produces the final readiness assessment and release recommendation.

6. Executable Dependency Graph

The orchestrator represents engineering work as a directed graph. A task becomes READY only when its dependencies, policy conditions and required artifacts are satisfied. Independent tasks execute in parallel and synchronize at explicit gates.

Example workflow: REQ-001 → PLAN-001 → [ARCH-001 + API-001 + SECURITY-001] → CODE-001 → [TEST-001 + REVIEW-001] → VALIDATE-001 → APPROVAL-001 → RELEASE-001.

Task contract:

- Task ID and type
- Dependencies
- Input artifact versions
- Expected output artifacts
- Assigned agent
- Required permissions
- Risk level
- Retry and timeout policy
- Approval policy
- Current status
- Decision references

7. Workflow State and Gates

Recommended states:

- PENDING
- READY
- RUNNING
- WAITING_APPROVAL
- COMPLETED
- FAILED
- RETRYING
- ROLLED_BACK
- BLOCKED
- CANCELLED

Each major lifecycle stage has an entry gate and exit gate. Entry gates verify that required inputs exist and policies allow execution. Exit gates verify artifact quality, required tests and downstream readiness before advancing the workflow.

8. Agent Permission Model

Agent	Read	Modify Code	DB Change	Deploy	Approval
Requirement	Requirements	No	No	No	No
Architecture	Code/design	No	No	No	No
Implementation	Code	Yes	No	No	No
Testing	Code/tests	Tests only	No	No	No
Security	Code/config	No	No	No	No
Release	Approved artifacts	No	No	No	Required

9. Human Approval and Controlled Autonomy

The system uses risk-based approval rather than requiring approval for every action. Low-risk engineering tasks can execute automatically. High-impact actions are paused at a human approval checkpoint.

- Production deployment
- Destructive database migration

- Security-sensitive change
- Access-control or permission change
- High-risk infrastructure or configuration change

Approval flow: Agent proposal → Risk evaluation → Policy decision → Human approval if required → Execute or reject/replan.

10. Artifact and Decision Lineage

The platform maintains a traceable relationship between requirements, decisions, tasks and artifacts.

Example:

Requirement v1 → Architecture v1 → API v1 → Code v1 → Tests v1

If Requirement v2 changes analytics retention, the impact engine identifies affected artifacts and tasks.

Unaffected completed work remains valid. Each new artifact version references the input versions and decision records that produced it.

11. Dynamic Replanning

1. Detect a change to a requirement or upstream artifact.
2. Create a new version and perform impact analysis.
3. Traverse dependency and artifact-lineage relationships.
4. Mark affected tasks stale or invalid.
5. Preserve unaffected completed work.
6. Generate a revised dependency graph.
7. Re-evaluate risk and approval requirements.
8. Resume execution from the earliest affected node.

This avoids restarting the entire SDLC whenever a requirement changes and directly demonstrates stateful, governed autonomy.

12. Recovery and Safe-Stop Strategy

Failure	Action	Escalation
Transient agent failure	Bounded retry with backoff	Fallback after retry limit
Tool timeout	Retry within timeout budget	Safe-stop if exhausted
Validation failure	Return to responsible task	Replan if repeated
Invalid high-impact change	Rollback checkpoint	Human review
Policy violation	Block execution	Security/human review
Unrecoverable failure	Safe-stop workflow	Human intervention

13. URL Shortener Application

The URL Shortener provides a concrete workload for demonstrating the platform. The application should include core APIs, analytics and reliability features.

- POST /api/v1/urls – create a short URL
- GET /{shortCode} – resolve and redirect
- GET /api/v1/urls/{shortCode}/analytics – retrieve analytics

Representative URL data: id, short_code, original_url, created_at, expires_at, created_by, status.

Representative analytics data: id, short_code, timestamp, ip_hash, user_agent, referrer.

14. Greenfield Scenario

Requirement: Build a URL Shortener from scratch with URL creation, redirection, analytics and reliability features.

9. Requirement Agent normalizes the requirement and identifies assumptions.
10. Planning Agent creates the dependency graph.
11. Architecture, API/Data and Security tasks execute in parallel where dependencies allow.
12. Implementation Agent generates the service.
13. Testing and Code Review execute in parallel.
14. Validation evaluates requirements, tests, security and quality.
15. Release readiness is produced and high-impact actions pause for approval.

15. Brownfield Scenario

Requirement: Add URL expiration to an existing URL Shortener without breaking current redirect behavior.

16. Codebase Analysis Agent discovers controllers, services, repositories, schema and tests.
17. Impact Analysis identifies affected modules and artifacts.
18. Planning Agent creates a minimal change graph.
19. Implementation modifies only affected components.
20. Regression tests validate existing behavior.
21. Validation and security gates determine release readiness.

16. Ambiguous Scenario

Requirement: Make the URL Shortener highly scalable and secure.

The system must not silently invent architecture targets. The Requirement Agent should identify missing measurable information such as expected traffic, availability target, geographic scope, authentication requirements, data retention and compliance constraints. It can ask for clarification or proceed only with explicitly recorded assumptions.

17. Engineering Output Generation

The platform should produce:

- Production-quality source code
- API definitions
- Database/schema definitions
- Unit tests
- Integration tests
- Security findings
- Architecture documentation
- Setup instructions
- Validation report
- Risk/trade-off analysis

- Release-readiness report

18. Validation Framework

- Requirement-to-task coverage
- Architecture/design consistency
- Build and compilation
- Unit and integration tests
- API contract validation
- Security validation
- Regression testing
- Policy compliance
- Artifact quality and completeness
- Release-readiness gate

19. Observability and Metrics

The orchestrator should provide audit-grade traceability and track at least:

- Workflow success rate
- Agent success/failure rate
- Retry frequency
- Rollback frequency
- Mean Time To Recovery (MTTR)
- End-to-end workflow latency
- Approval wait time
- Replanning count
- Blocked/safe-stop count

20. Security, Compliance and Change-Control Guardrails

- Validate and constrain URL input.
- Apply rate limiting and abuse controls.
- Protect analytics access.
- Prevent agents from accessing or exposing secrets.
- Block prohibited or destructive operations.
- Enforce approval for high-impact changes.
- Record policy decisions and security findings in the audit trail.

21. Technology Stack

- Java + Spring Boot
- PostgreSQL for durable application and workflow state
- Redis for URL-resolution caching where appropriate
- JUnit, Mockito and Testcontainers
- Docker
- OpenAPI
- Structured logging and metrics
- LLM provider supporting structured tool/function calling

22. Proposed Project Structure

agentic-platform/	
├── orchestrator/	WorkflowEngine, GraphEngine, Scheduler, RecoveryManager
├── agents/	Requirement, Planning, Architecture, Coding, Testing, Security
├── policy/	PolicyEngine, RiskEngine, PermissionModel
├── lineage/	ArtifactStore, VersionManager, DecisionLineage
├── workflow/	WorkflowState, TaskState, WorkflowRepository
├── replanning/	ImpactAnalyzer, GraphReplanner
├── audit/	AuditService, EventPublisher
└── url-shortener/	controller, service, repository, model, configuration

23. Key Engineering Trade-offs

Decision Area	Recommended Decision
Microservices vs modular monolith	Use a modular Spring Boot prototype for the 2–3 day assessment; preserve clear domain boundaries.
Autonomy vs safety	Allow bounded autonomous execution; require approval for high-impact actions.
Restart vs replanning	Use impact-based replanning to reduce cost and regression risk.
LLM flexibility vs deterministic control	Use structured outputs, schemas, validation and explicit state transitions.
Automation vs engineering judgment	Automate repeatable engineering work while keeping humans responsible for critical decisions.

24. Demo Plan

1. Submit the greenfield URL Shortener requirement.
2. Show normalized requirements, assumptions and acceptance criteria.
3. Display the executable dependency graph.
4. Demonstrate parallel architecture/API/security execution.
5. Show generated implementation and tests.
6. Introduce a controlled validation failure and demonstrate retry/recovery.
7. Trigger a high-impact operation and demonstrate human approval.
8. Change an upstream requirement and demonstrate targeted dynamic replanning.
9. Show artifact versions and decision lineage.
10. Show reliability metrics and audit history.
11. Run the brownfield and ambiguous scenarios.
12. Present the final engineering package.

25. Assessment Success Criteria

- Runnable end-to-end prototype.
- Clear architecture and orchestration model.
- Executable dependency graph with entry/exit gates.
- Non-linear sequential and parallel execution.
- Persistent context and decision lineage.
- Human approval for high-impact actions.
- Bounded retry, fallback, rollback and safe-stop.
- Security, compliance and change-control guardrails.
- Audit-grade observability and reliability metrics.
- Dynamic replanning after upstream changes.
- Production-quality engineering outputs.
- Demonstrated greenfield, brownfield and ambiguous scenarios.
- Clear risks, trade-offs, assumptions and limitations.

26. Limitations and Assumptions

The assessment provides the required scenario and capabilities but does not prescribe a specific LLM provider, workflow framework, database, deployment platform or exact API contract. Those details should therefore be treated as implementation choices rather than assessment-mandated requirements.

For a 2–3 day prototype, the recommended implementation should prioritize demonstrable orchestration, governance and engineering judgment over unnecessary infrastructure complexity.

27. Final Engineering Summary

The solution treats the Agentic Software Engineering System as the primary product and the URL Shortener as its demonstration workload. The core capability is a governed, stateful execution engine that coordinates specialized agents through an explicit dependency graph.

The strongest differentiators are executable graph orchestration, versioned artifact lineage, explicit agent permissions, risk-based human approval, bounded recovery, auditability and impact-based dynamic replanning. Together these capabilities provide a defensible demonstration of controlled autonomy and end-to-end SDLC automation.